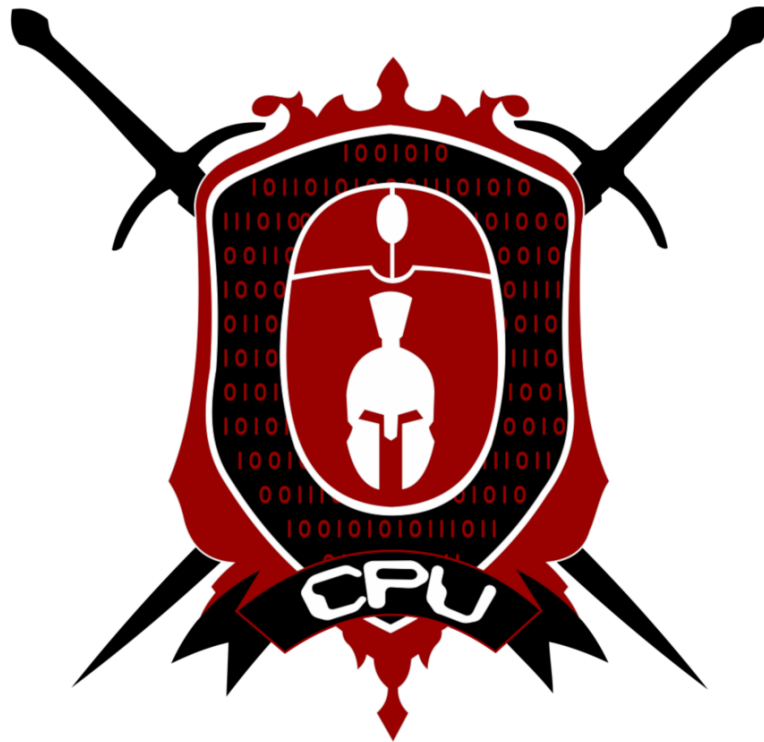




```
#ifndef __PROGBASE_2023__  
#define __PROGBASE_2023__
```

A) Sobre a entrada:

- 1) A entrada de seu programa deve ser lida da entrada padrão.
- 2) A entrada é composta por vários casos de teste, cada um descrito em um número de linhas que depende do problema.
- 3) Quando uma linha da entrada contém vários valores, estes são separados por um único espaço em branco; a entrada não contém nenhum outro espaço em branco.
- 4) Cada linha, incluindo a última, contém o caractere final-de-linha.
- 5) O final da entrada coincide com o final do arquivo.

B) Sobre o seu código:

- 1) O nome do arquivo de seu programa é o nome do problema seguido da extensão da linguagem escolhida.
- 2) O seu programa deve ser escrito em uma das linguagens permitidas: Java, C, C++ ou Python 2 ou 3.
- 3) Um problema pode ser resolvido em uma linguagem diferente de outro, se desejar.
- 4) O seu programa deve ler a entrada do console (entrada padrão).
- 5) O seu programa deve gerar a saída em console (saída padrão).

C) Sobre a saída:

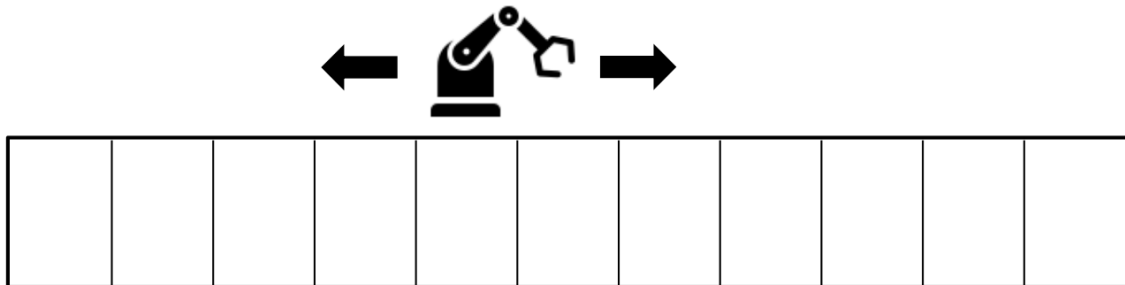
- 1) Quando uma linha da saída contém vários valores, estes devem ser separados por um único espaço em branco; a saída não deve conter nenhum outro espaço em branco.
- 2) Cada linha, incluindo a última, deve conter o caractere final-de-linha.

D) Sobre a competição:

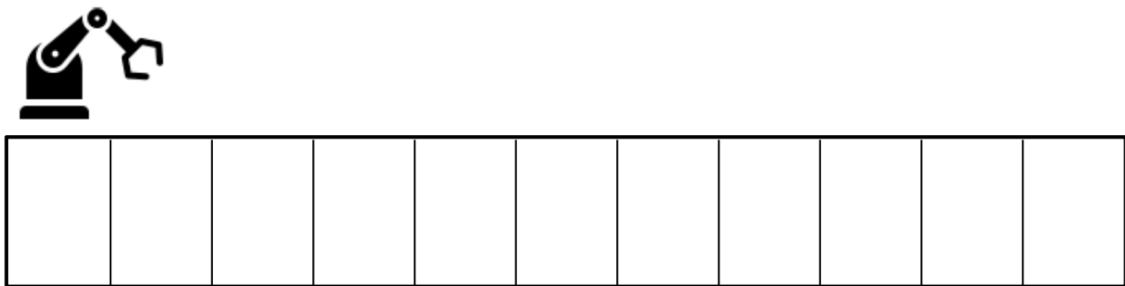
- 1) Linguagens aceitas:
 - a) C, extensão “.c” → `gcc -std=c99 -O2 -lm`
 - b) C++, extensão “.cpp” → `g++ -std=c++17 -O2 -lm`
 - c) Java, extensão “.java” → `LC_ALL=en_US.UTF-8 java .. +2s` para limite de tempo
 - e) Python 3, extensão “.py” → `python3 .. +1s` para limite de tempo
- 2) As questões, embora escritas de forma original pela equipe organizadora, podem ser baseadas em problemas de mesma categoria já existentes.

Problema A
Autômato Zigue-Zague
Arquivo: automato.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Thiago Maia & Ana Marreiros Maia

O Problema: Um autômato de fábrica foi construído para fazer dois tipos de movimento: avanço (uma única casa para a direita) e retrocesso (uma única casa para a esquerda).



O autômato sempre parte de um ponto inicial e deve retornar a esse ponto ao final de qualquer sequência de movimentos.



Início

Seguindo da ilustração imediatamente acima, o primeiro movimento sempre deve ser para a direita, e o autômato não pode ser movimentado em um sentido mais vezes que no outro. Também seguindo da segunda ilustração, o espaço de movimentação do autômato é limitado àquele total de onze casas. A você foi solicitado para que, dada uma sequência de operações, indicar se ela é viável ou não.

Entrada:

A entrada consiste de um conjunto de sequências de movimentos a serem executados pelo autômato, uma sequência em cada linha, sendo que as movimentações para a esquerda são representadas pela letra 'E', e as movimentações para a direita por 'D'. O máximo são 100.000 movimentações por sequência de operações do autômato. O final das sequências coincide com o final da entrada.

Saída: Para cada sequência de entrada deve haver em uma linha única a resposta 'S' ou 'N' indicando se esta é viável ou não, para o autômato, conforme as regras.

Exemplo de entrada:

EDEDDE
DDDEEE
DEDEDE
DDE

Saída do exemplo de entrada:

N
S
S
N

Problema B
Bêbado ao Volante
Arquivo: bebado.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Hamilton José Brumatto

O Problema:

Criado em 1997, o Código de Trânsito Brasileiro teve como uma de suas principais diretrizes trazer segurança para o tráfego e diminuir o número de vítimas das ruas e estradas. Um dos meios para atingir esse objetivo foi atacar uma das grandes causas dos acidentes: a bebida no volante. A lei estabeleceu tolerância de 0,06% de álcool no sangue. Uma concentração superior rendia uma infração gravíssima com multa multiplicada por cinco, suspensão do direito de dirigir e recolhimento do veículo. Em 2006 foi introduzida a prova testemunhal do agente de trânsito e se tornou crime causar a morte de alguém em um acidente após consumir álcool. Em 2008 veio a norma que passou a ser conhecida como lei seca, que adotou a tolerância zero ao álcool na direção e criminalizou a infração de dirigir embriagado – com pena de 3 a seis meses de prisão. Quatro anos depois, a multa para beber e dirigir dobrou, passando para R\$ 2.934 reais - paga ainda em dobro no caso de reincidência em 12 meses -, e passaram a ser aceitas como provas de embriaguez imagens e vídeos. Em 2016 outra mudança foi feita para tratar a recusa do teste como uma infração em si, sujeita a todas as penas de ser flagrado embriagado na direção. Em 2018 foram ainda aumentadas as punições para quem dirigir bêbado e provocar uma morte, para até oito anos de prisão, e para quem causar ferimentos graves, até cinco anos. A você foi pedido um algoritmo onde dado o índice de álcool no sangue indique se o motorista foi flagrado dirigindo alcoolizado ou está livre para seguir viagem.

Entrada:

Cada caso de entrada representa um caso de teste. A entrada é representada um valor de ponto flutuante representando o índice percentual de álcool no sangue, com duas casas decimais.

Saída:

Para cada caso de teste deve-se imprimir em uma linha única a decisão do oficial de trânsito para o motorista, que pode ser: “FLAGRADO ALCOOLIZADO”, ou “LIVRE” (sem aspas).

Exemplo de entrada:

Saída do Exemplo de entrada:

0.01%	LIVRE
0.10%	FLAGRADO ALCOOLIZADO
0.06%	LIVRE

Problema C
Carta Criptografibonacciada
Arquivo: carta.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Thiago Maia & Ana Marreiros Maia

O Problema:

Um grupo de três amigos iam fazer intercâmbio em países diferentes, e planejaram trocar cartas enquanto distantes. Como não queriam que outras pessoas soubessem do teor das conversas, resolveram criar um esquema de criptografia baseado na sequência de Fibonacci. Sendo $O(\text{letra})$ a posição original do caractere 'letra' no alfabeto latino (composto por 26 letras), ou seja, $O(a) = 1$, $O(b) = 2$, ..., $O(z) = 26$, e sendo $M(\text{letra})$ a posição do caractere 'letra' na mensagem a ser criptografada. Por exemplo, na mensagem 'alo mundo!', $M(a) = 1$, $M(l) = 2$, $M(o) = 3$, $M(m) = 5$, ..., $M(o) = 9$. Nesse esquema de criptografia, criado por eles, cada letra ' l ' da mensagem original deve ser trocada por uma outra letra $C(x)$, cuja posição no alfabeto é determinada por x . Para calcular x tem-se:

$$x = O(l) + \text{Fib}(M(l))$$

onde $\text{Fib}(i)$ corresponde ao i -ésimo elemento na sequência de Fibonacci. Vale lembrar que, como o alfabeto contém 26 letras, este deve ser tratado em círculo se necessário, ou seja, a posição $C(27) = 'a'$, $C(28) = 'b'$ e assim por diante. Os caracteres de pontuação e tabulação não são criptografáveis, mas suas posições contam!

Entrada:

A primeira linha da entrada deve conter um caractere 'C' ou 'D', informando se a mensagem deve ser criptografada ou decriptografada, respectivamente. Na segunda linha estará contida a mensagem-alvo, composta apenas por caracteres minúsculas e de no máximo 200 caracteres. Existem pontuações, mas não há caracteres acentuados.

Saída: A mensagem resultado.

Exemplo de entrada:

```
C
alo mundo!
```

Saída do Exemplo de entrada:

```
bmq rcayw!
```

```
D
cpc xweom!
```

```
boa sorte!
```

Problema D

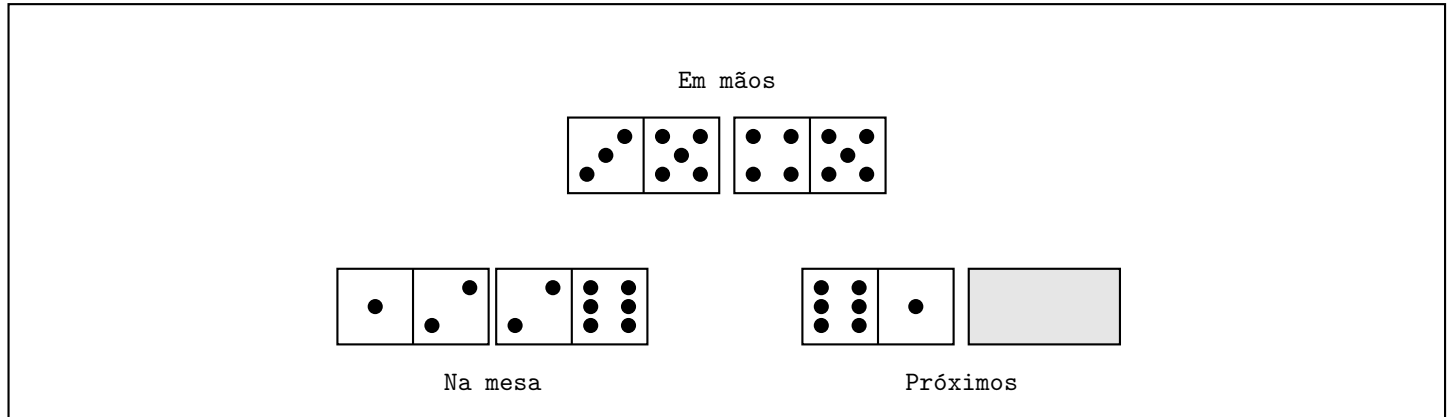
Dominó Oculto

Arquivo: *domino*.*[c|cpp|java|py]*

Limite de tempo: 3 s

Autor: Hamilton José Brumatto

O Problema: O jogo de dominó é viciante, tanto que até se inventam variantes do jogo, até mesmo para uma pessoa. Um exemplo é a paciência com Dominó Oculto. Ao jogador é sorteada N peças de dominó, e ele irá abrindo uma por uma de forma oculta, a primeira é colocada na mesa, sempre que encontra uma peça que caiba na mesa, seguindo a regra do jogo de dominó, esta deve ser colocada, quando não cabe, fica guardada como pedra na mão. As pedras nas mãos quando cabem devem também ser colocadas, antes da retirada da próxima pedra oculta. A principal dúvida é se a versão final que foi criada na mesa, após tomar todos os dominós ocultos é a maior possível. Vamos considerar a seguinte situação:



Na sequência da jogada podemos ficar com 1 nas duas pontas ou 6 nas duas pontas. A jogada deve ser feita. Vamos considerar a que próxima peça fosse $\{3, 6\}$, se tivéssemos ficado com 1 nas duas pontas, o jogo teria acabado com 3 peças na mesa e 3 nas mãos. Mas se tivéssemos feito 6 nas duas pontas, aí conseguiríamos encaixar a peça $\{3, 6\}$, ficando com 3 e 6 nas pontas, aí encaixaria a peça $\{3, 5\}$, ficando com 5 e 6 nas pontas e por fim a peça $\{4, 5\}$, concluindo a jogada: 6 peças na mesa e nenhuma nas mãos. A você foi solicitado um algoritmo que retorne qual a maior sequência possível na mesa seguindo uma ordem de pedras ocultas.

Entrada: São vários casos de teste, cada caso de teste possui duas linhas, a primeira linha N , ($0 < N \leq 28$) o número de peças ocultas, e na segunda linha são N pares de valores $\{v_i, v_j\}$ separados por espaço, onde $0 \leq v \leq 6$ e cada par é único (não importando a ordem). Os casos de teste terminam com $N = 0$, que não deve ser considerado.

Saída: Para cada caso de teste deve haver em uma linha exclusiva um número inteiro indicando a quantidade máxima de dominós que pode-se colocar na mesa.

Exemplo de entrada:

```
6
{1,2} {3,5} {4,5} {2,6} {1,6} {3,6}
2
{3,4} {2,6}
1
{0,0}
0
```

Saída do exemplo de entrada:

```
6
1
1
```

Problema E

Enigma

Arquivo: *enigma*.*[c|cpp|java|py]*

Limite de tempo: 1 s

Autor: Hamilton José Brumatto

O Problema:

As crianças são muito inventivas em seus jogos. Principalmente quando se trata da linguagem falada ou escrita. Na linguagem falada um exemplo é a língua do “P”, mas o mais intrigante é na linguagem escrita. As crianças sabem que os pais (zelosos) monitoram tudo que elas escrevem ou recebem por escrito. Então já aprendem, desde cedo, “criptografar” suas mensagens. Um exemplo é o jogo: ZENIT-POLAR. Esta é uma criptografia por substituição: a letra Z é trocada nos textos pela letra P e vice-versa e assim as letras sucessivas do jogo. A seguir um exemplo de uma frase em ZENIT-POLAR:

“Osri citri olagmiraci lie o ziti quinquot um not”

Podemos traduzir facilmente conhecendo o código: ”Esta carta enigmatica nao e para qualquer um ler”. Só que para ser mais rápido foi pedido a você um algoritmo que dada uma frase a traduza. Observe que a frase pode estar no português para traduzir para o ZENIT-POLAR, ou em ZENIT-POLAR, para traduzir para o português.

Entrada:

A entrada apresenta várias linhas cada linha contém um texto não acentuado com caracteres ASCII padrão. Cada linha tem tamanho máximo de 100 caracteres. A entrada termina com o final do arquivo.

Saída:

Para cada linha de entrada deve haver uma linha na saída com o texto traduzido respeitando-se maiúsculas e minúsculas.

Exemplo de entrada:

```
Osri citri olagmiraci lie o ziti quinquot um not
Quero cifrar um texto usando ZENIT-POLAR
```

Saída do exemplo de entrada:

```
Esta carta enigmatica nao e para qualquer um ler
Quote caftit um roxre usilde POLAR-ZENIT
```

Problema F

Frete Certo

Arquivo: *frete*.*[c|cpp|java|py]*

Limite de tempo: 1 s

Autor: Hamilton José Brumatto

O Problema: A “Compras Por Oportunidade” também conhecida como CPU é uma rede varejista online. Ela tem lojas espalhadas em vários cantos do seu país de origem o “Estados Republicanos do BARBO-SERVOS” que vamos apelidar de ERBASE. O ERBASE é um país insular, onde há pontes ligando as várias Ilhas-Estados. As pontes são de mão dupla e não possuem interseção, exceto nas ilhas. Não há grupos isolados de Ilhas, todas estão interligadas. O presidente recém eleito resolveu retomar vários impostos que haviam sido extintos, para bancar seu séquito, a exemplo do pedágio-ponte, todas pontes têm pedágios, nos dois sentidos. O fato é, a CPU quer oferecer o menor custo de frete para uma compra realizada, e este custo é basicamente o valor dos pedágios transpostos entre a saída do produto de uma ilha e a entrega na ilha do comprador. Temos então a lista de Ilhas-Estados do ERBASE e identificamos quais Ilhas-Estados possui sede da CPU. Temos também o conjunto de pontes e o valor do pedágio em cada ponte. Por fim, dada a ilha do comprador, e a lista de ilhas das unidades da CPU que possui o produto, queremos saber o menor custo do frete. A você foi pedido que criasse um algoritmo que aponte qual a unidade da CPU que terá o menor custo de frete para entrega do produto e o valor deste frete.

Entrada:

O caso de teste começa com dois números N e M indicando o número de Ilhas-Estados ($0 < N \leq 100$), que numeramos de 1 a N , para evitar o uso de nomes, e o número de pontes que ligam estas Ilhas ($N - 1 \leq M < N^2$). Em seguida são M linhas com as seguintes informações: dois números A e B indicando duas ilhas ($0 < A \neq B \leq N$); P , ($0 < P \leq 100$) o número de moedas-pedágio que é o custo para atravessar a ponte que liga as ilhas A e B . Em seguida temos uma linha com L , ($0 < L \leq N$) o número de unidades da CPU na ERBASE, seguido de L inteiros distintos no intervalo de 1 a N indicando em quais ilhas há uma unidade da CPU. Por fim temos Q ($0 < Q \leq 100$) o número de compras realizadas seguidas de Q linhas com as seguintes informações: C , ($0 < C \leq N$) a ilha do comprador, D , ($0 < D \leq L$) o número de unidades que possui o produto comprado e uma sequência de D números que são um subconjunto dos L valores, representando as unidades que possuem o produto. É garantido que somente uma unidade irá oferecer o menor custo de entrega.

Saída: A saída deve conter Q linhas onde cada linha contém dois números, o número da ilha onde está a unidade da CPU que irá realizar a entrega e o custo em moedas-pedágio para a entrega.

Exemplo de entrada:

```
11 15
9 8 4
10 7 2
10 11 3
3 4 3
4 5 1
11 5 2
1 2 2
2 10 4
10 9 1
7 11 1
2 3 1
3 11 2
6 7 3
6 11 4
7 8 2
4 1 10 4 6
5
11 2 1 4
8 3 6 4 1
7 2 10 6
4 3 1 6 4
8 4 1 4 6 10
```

Saída do exemplo de entrada:

```
4 3
6 5
10 2
4 0
10 4
```


Problema G
Um Grau no Asfalto
Arquivo: grau.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Hamilton José Brumatto

O Problema: A cidade de Cafundós do Porto Úmido, carinhosamente apelidada de CPU tem uma característica semelhante à turística cidade de Canavieiras/BA: seu prefeito é um simpático motociclista. A cidade possui vários distritos que são interligados todos por estrada de chão. O prefeito, agora com um Harlão de seus sonhos, percebeu que não é tão agradável por a moto na estrada de chão. Ele tomou a decisão de que irá dar um grau na cidade asphaltando as estradas que ligam os vários distritos, no entanto, ele quer economizar. Ou seja, é necessário que de um distrito para qualquer outro, exista um caminho asphaltado, mas isto tem de acontecer asphaltando o mínimo necessário, nem todas as estradas serão asphaltadas. Ele pediu para você para que dado o conjunto de estradas e o comprimento de cada estrada, que faça um algoritmo para determinar quantos quilômetros precisam ser asphaltados no mínimo.

Entrada:

O caso de teste começa com dois números N e M indicando o número de Distritos ($0 < N \leq 100$), que numeramos de 1 a N , para evitar o uso de nomes, e o número de estradas que ligam estes distritos ($N - 1 \leq M < N^2$). Em seguida são M linhas com as seguintes informações: dois números A e B indicando dois distritos ($0 < A \neq B \leq N$); K , ($0 < K \leq 100$) a distância em quilômetros da estrada que liga os distritos A e B .

Saída:

A saída deve conter em uma única linha um inteiro indicando o tanto de quilômetros que deve ser asphaltado, no mínimo, para ter todos os distritos ligados por um caminho asphaltado.

Exemplo de entrada:

```
11 15
9 8 4
10 7 2
10 11 3
3 4 3
4 5 1
11 5 2
1 2 2
2 10 4
10 9 1
7 11 1
2 3 1
3 11 2
6 7 3
6 11 4
7 8 2
```

Saída do exemplo de entrada:

```
17
```

Problema H

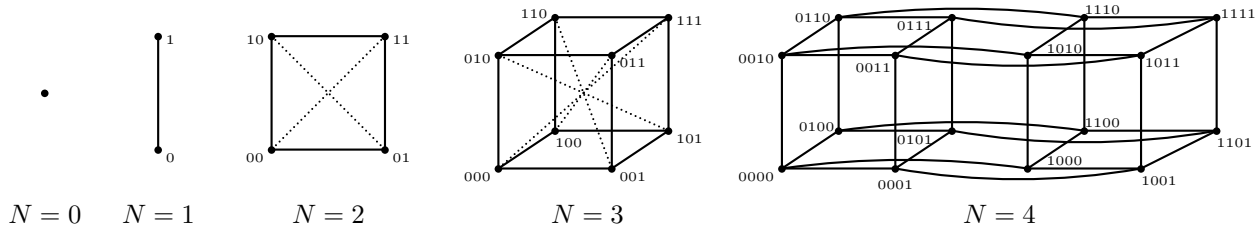
Hiper-Cubo

Arquivo: *hipercubo*.*[c|cpp|java|py]*

Limite de tempo: 1 s

Autor: Hamilton José Brumatto

O Problema: Um Hiper-Cubo é um cubo no espaço vetorial de dimensão N . As figuras abaixo mostram os cubos e suas arestas nos espaços $N = 0$ (uma abstração matemática já que todo o espaço vetorial está restrito a um ponto), $N = 1$ (o espaço linear), uma reta, $N = 2$ (o espaço bidimensional), um quadrado, $N = 3$ (o espaço tridimensional), o cubo tradicional, $N = 4 \dots$



Sobre os hipercubos há uma curiosidade, um hipercubo de dimensão N tem seus vértices classificados, ou enumerados, a partir de um conjunto de N bits como podemos ver na figura acima. Em especial, as arestas, desenhadas por uma linha contínua, são representadas pela ligação de 2 vértices, que distam entre si por apenas 1 bit. As diagonais principais, desenhadas por linhas pontilhadas (omitida para $N = 4$), são representadas pela ligação de 2 vértices onde todos os bits são trocados. As demais diagonais secundárias, terciárias, ... possui a distância entre os bits dos vértices no intervalo $(1, n)$ bits. A você foi pedido, dada a dimensão N do espaço ($0 \leq N \leq 20$) que indique quantos vértices, quantas arestas, quantas diagonais principais e quantas diagonais não principais o hipercubo possui.

Entrada:

Cada caso de teste é dado por um número inteiro N , $0 \leq N \leq 20$. Os casos de teste terminam com as entradas.

Saída:

Em cada linha, para cada caso de teste, deve ter 4 números indicando respectivamente: O número de vértices do hipercubo, o número de arestas, o número de diagonais principais e o número de diagonais não principais.

Exemplo de entrada:

```
0
1
2
3
4
```

Saída do exemplo de entrada:

```
1 0 0 0
2 1 0 0
4 4 2 0
8 12 4 12
16 32 8 80
```

Problema I
Ilhas Isoladas
Arquivo: ilhas.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Hamilton José Brumatto

O Problema: Voltamos aos Estados Republicanos do BARbo-Servo (ERBASE), infelizmente a natureza foi cruel e um grande maremoto derrubou várias das pontes. Principalmente pelo fato de que haviam sido construídas em um esquema fraudulento com as construtoras em uma época passada. As pontes não eram de prima qualidade e muitas delas ruíram. A grande preocupação é saber se alguma ilha ficou isolada, ou seja, se há ilhas que não tenham mais nenhuma ligação com qualquer outra. Pois estas é quem receberão os primeiros socorros. A você foi solicitado que, dadas as ligações entre as ilhas que restaram que fizesse um algoritmo para saber se há ilhas isoladas.

Entrada:

A entrada consiste, na primeira linha, em dois valores: N , $1 \leq N \leq 100$ que representa o número de ilhas da ERBASE, as ilhas são numeradas para evitar o uso de nomes na indexação, e M , $0 \leq M \leq (N^2 - N)/2$ que consiste na quantidade de pontes que restaram após a destruição, e em seguida são M linhas contendo dois valores a e b $1 \leq a, b \leq N$, separados por um espaço em branco, sendo que a b são duas ilhas distintas ainda ligadas por uma ponte.

Saída:

A saída deve conter em uma única linha para cada caso de teste um caracter ‘S’ indicando que há ilhas isoladas ou ‘N’ indicando que não há.

Exemplo de entrada:

```
11 14
9 8
10 7
10 11
3 4
4 5
11 5
2 10
10 9
7 11
2 3
3 11
6 7
6 11
7 8
6 4
1 2
2 3
4 5
5 6
0 0
```

Saída do exemplo de entrada:

```
S
N
```

Problema J
Jobs em Ordem
Arquivo: jobs.[c|cpp|java|py]
Limite de tempo: 1 s
Autor: Hamilton José Brumatto

O Problema:

Todos os dias diversos estudantes mundo afora fazem as suas listas “TO-DO”, com o intuito de organizar as seus *jobs* de maneira que nada deixe de ser feito. Porém, organizar a ordem em que os *jobs* devem ser executados nem sempre é fácil, uma vez que alguns *jobs* têm que ser executados antes de outros, e não saber nem por onde começar leva muita gente a procrastinar e até desistir das obrigações! Na tentativa de organizar isso no seu caso, a Lara desenvolveu um esquema de enumeração dos *jobs* em uma ordem em que ela desejaria executar, sem quebrar requisito (justo sem se preocupar com a dependência de ordem entre os *jobs*). Em seguida ela criou uma lista de pares ordenados dos *jobs*, indicando qual *job* tem prioridade sobre outro. O que ela deseja fazer então é estabelecer uma ordem em que os *jobs* possam ser executados a contento, e pediu a você que crie um algoritmo para organizar os *jobs*.

Entrada:

A entrada consiste, na primeira linha, em dois valores: N , $1 \leq N \leq 100$ que representa o número de *jobs* na lista, e M , $0 \leq M < N^2$ que consiste na quantidade de pares ordenados de dependência entre os *jobs*, e em seguida são M linhas contendo dois valores a e b $1 \leq a, b \leq N$, separados por um espaço em branco, sendo que a representa um *job* que deve ser executado antes do job b . Sabe-se que não há dependências circulares entre os *jobs*, ou seja, sempre é possível obter uma ordenação dos *jobs* que satisfaça as dependências.

Saída:

Para cada lista de *jobs*, apresentar uma ordem de execução plausível dos mesmos que portanto atenda às necessidades da Lara.

Exemplo de entrada:

Saída do Exemplo de entrada:

```
9 11
1 3
3 4
3 5
3 9
5 2
9 8
6 1
6 8
7 2
7 5
8 7
```

```
6 1 3 9 8 7 5 2 4
```

Problema K

Kelvin, Celsius e Fahrenheit

Arquivo: *kelvin*.*[c|cpp|java|py]*

Limite de tempo: 1 s

Autor: Hamilton José Brumatto

O Problema: A noção de quente e frio, falando sobre temperatura, sempre esteve presente entre nós, tanto que urgiu medir a temperatura. Vários pioneiros criaram um sistema de medidas. Os primórdios das medidas veio com Isaac Newton, por volta de 1700, criou uma escala forma qualitativa, compreendendo cerca de 20 pontos de referência que variavam de “ar frio no inverno” para “brasas no fogo da cozinha”, mas se sentiu insatisfeito e tomou dois pontos o derretimento da neve 0 *N* e a ebulição da água 33 *N*, sendo a precursora da escala Centígrada ou Celsius. Na mesma época Ole Christensen Rømer criou a escala com o congelamento da salmoura 0 *Rø* e ebulição da água 60 *Rø*. Daniel Gabriel Fahrenheit em 1724 criou sua escala de temperatura, definindo o 0 *F* uma mistura de água, gelo picado, sal e amônia e o 100 *F* que foi a temperatura do corpo de sua mulher (ela estava febril). Uma grande contribuição de Fahrenheit foi o termômetro de mercúrio. O grande problema desta escala adotada em países de língua inglesa, é a definição formal dos pontos fixos. Anders Celsius em 1742 publicou um trabalho “Observações sobre 2 graus persistentes em um termômetro”, que são o ponto de fusão do gelo e o ponto de evaporação da água, e tabelou-os como 0 e 100. Por ser baseada em centésimos, a escala ganhou o nome de Centígrada, mas futuramente foi nomeada em homenagem a seu criador: Celsius. Também tem problemas com a definição formal, pois estas temperaturas dependem da pressão atmosférica e acabou sendo adotada pelo sistema métrico. Em paralelo René-Antoine Ferchault de Réaumur criou sua escala o mesmo 0, mas a temperatura de ebulição ficou como 80 *R*. Por fim veio uma escala científica, como a Kelvin, em homenagem ao físico e engenheiro irlandês William Thomson (1824–1907), 1º barão Kelvin, que em 1848 escreveu sobre a necessidade de uma escala absoluta. Esta escala toma como 0 a temperatura 0 absoluta, onde não há mais movimento de átomos (já que a temperatura vem da agitação dos átomos), e adotou-se a temperatura tripla da água como 273.16*K*, isto feito para que a escala Kelvin e Celsius tenha a mesma largura de uma unidade. A temperatura tripla da água na escala Celsius é 0.01 *C*, e o zero absoluto $-273.15\text{ }^{\circ}\text{C}$. Por fim o engenheiro e físico escocês William John Macquorn Rankine propôs em 1859 a escala Rankine, também absoluta, mas com a diferença de grau igual à da escala Fahrenheit, 0 *Ra* = $-459.67\text{ }^{\circ}\text{F}$. No passado adotava-se o símbolo de grau ‘°’ antes da unidade da escala mas isto tornou-se obsoleto. Abaixo está uma tabela de conversões, com base na escala Kelvin. A você foi solicitado um algoritmo que realize as conversões.

Escala	Fórmula
Fahrenheit	$F = K \times 1.8 - 459.889$
Celsius	$C = K - 273.15$
Rankine	$Ra = K \times 1.8$
Réaumur	$Re = (K - 273.15) \times 0.8$
Rømer	$Ro = (K - 273.15) \times 21/40 + 7.5$

Entrada:

A entrada é um caso de teste de duas linhas, a primeira linha é um texto representando um evento com no máximo 100 caracteres UTF-8 e a segunda linha é a temperatura do evento, em ponto flutuante, em uma determinada escala, que pode ser: *K* - Kelvin, *F* - Fahrenheit, *C* - Celsius, *Ra* - Rankine, *Re* - Réaumur ou *Ro* - Rømer. A unidade está separada da medida por um espaço em branco.

Saída:

A saída deve conter seis linhas onde cada linha temos temperatura do evento em cada uma das seis escalas, nesta ordem: *K*, *F*, *C*, *Ra*, *Re*, *Ro*. Todas com duas casas decimais. A unidade deve estar separada da medida por exatamente um espaço em branco.

Exemplo de entrada:

Temperatura de fusão do Mercúrio
234.32 K

Saída do Exemplo de entrada:

234.32 K
-38.11 F
-38.83 C
421.78 Ra
-31.06 Re
-12.89 Ro

Temperatura da mulher de Fahrenheit
100 F

311.05 K
100.00 F
37.90 C
559.89 Ra
30.32 Re
27.40 Ro